Java Programming: Collections Framework

Study notes for a document-based Retrieval-Augmented Generation (RAG) project

How to use these notes

These notes provide connected explanations, examples, terminology, and review questions. They are intended as a starter corpus for testing PDF extraction, chunking, embedding, and retrieval. Read the sections and ask questions whose answers appear in the material.

1. Why collections?

The Java Collections Framework provides interfaces and classes for storing and processing groups of objects. It offers common data structures and algorithms so developers do not need to implement every list, set, or map from scratch. The framework is primarily in java.util.

2. Main interfaces

Collection is a root interface for groups of elements. List is an ordered collection that permits duplicates. Set does not permit duplicate elements. Queue represents elements waiting for processing. Map stores key-value associations and is not a subtype of Collection.

3. ArrayList

ArrayList is a resizable-array implementation of List. It provides fast indexed access and is convenient when elements are frequently read by position. Inserting or removing elements near the beginning or middle can require shifting later elements. ArrayList permits duplicates and preserves insertion order.

Code: ArrayList

```
import java.util.ArrayList;

class Example {
public static void main(String[] args) {
ArrayList<String> names = new ArrayList<>();
names.add("Maya");
names.add("Ravi");
names.add("Maya");
System.out.println(names.get(0));
System.out.println(names.size());
}
}
```

4. LinkedList

LinkedList is a doubly linked list implementation of List and Deque. It supports operations at both ends. Accessing an element by index requires traversal, so it is generally not the best choice when frequent random indexed access is required. Choose a collection based on the operations the application performs most often.

5. HashSet and TreeSet

HashSet stores unique elements and uses hashing to support efficient basic operations on average. It does not promise iteration order. TreeSet stores unique elements in sorted order according to natural ordering or a supplied Comparator. Maintaining sorted order has additional costs compared with a hash-based set.

6. HashMap

HashMap stores key-value pairs. Keys are unique; adding a value for an existing key replaces the previous value. HashMap permits one null key and null values. It does not guarantee iteration order. Keys should have stable equals

and hashCode behavior while stored in the map.

Code: HashMap

```
import java.util.HashMap;

class Example {
public static void main(String[] args) {
HashMap<String, Integer> marks = new HashMap<>();
marks.put("Anu", 90);
marks.put("Bala", 85);
System.out.println(marks.get("Anu"));
System.out.println(marks.containsKey("Bala"));
}
}
```

7. Iteration

The enhanced for loop is convenient for reading each element in an iterable collection. An Iterator provides controlled traversal and supports removal through its own remove method when permitted. Modifying many collections structurally while iterating outside the iterator can cause ConcurrentModificationException.

8. Generics

Generics allow classes and methods to operate on types while retaining compile-time type checks. List<String> indicates a list of strings. Generics reduce unsafe casts and make APIs clearer. Java generics use type erasure, so generic type parameters are generally not available as ordinary runtime class information.

9. Comparable and Comparator

Comparable defines a type's natural ordering through compareTo. Comparator defines an external ordering through compare. A Comparator is useful when objects need several possible sort orders, such as sorting employees by name or salary.

10. Choosing a collection

Use ArrayList for general-purpose ordered lists with frequent indexed reads. Use HashSet when uniqueness and fast membership checks matter but order does not. Use TreeSet when sorted unique elements are required. Use HashMap for key-value lookup. Use ArrayDeque for many queue and stack use cases.

11. Common mistakes

Do not assume HashMap or HashSet iteration order. Do not modify a collection in an unsafe way during enhanced-for iteration. Remember that List indexes start at zero. Use equals for logical object equality rather than == when comparing most object values. Consider null handling explicitly.

12. Review questions

1. How does List differ from Set?

2. Does HashMap preserve insertion order?

3. When is TreeSet useful?

4. What problem do generics solve?

5. What is the purpose of Comparator?